THE

pure-dango

PROGRAMMING LANGUAGE

Hans Jacob

Table of Contents

Introduction 3
Chapter 1 - Language Basics 4
1.1 – Hello, World! 4
1.2 – print 5
1.3 – Variables 6
1.3.1 – new 6
1.4 – Operators 7
1.4.1 – Binary Operators 7
1.4.2 – Unary Operators 8
1.4.3 – Assignment Operators 9
1.5 – Loops 10
1.5.2 – While Loop 10
1.7 – Answer Keys 11
Subtraction 11
Assignment 12

Introduction

pure-dango is a dynamically typed programming language built from JavaScript/TypeScript made for mathematical precision. It has functions specially built for doing decimal Math and general use.

pure-dango was created because most programming languages handle big numbers poorly. JavaScript for example, JavaScript shortens big numbers to scientific notation unless you use BigInt or an external library like Decimal.js. pure-dango solves this natively.

Chapter 1 - Language Basics

Welcome to the first chapter of pure-dango! My aim is to show the essential elements in building real programs in pure-dango without over-explaining it. This chapter is to give a basic tutorial on the types, keywords, and data types of the language.

The more detailed explanations will begin in Chapter 2.

1.1 – Hello, World!

The only way to learn a programming language is by experimenting and writing programs. The first program is the same in every language:

Print the words

```
Hello, World!
```

This is the first problem anyone overcomes in programming. You have to find somewhere to write the code, compile the code, run it, and find where your output went. Once you solve this, everything that follows becomes easier.

In pure-dango, the program to print `“Hello, World!”` is

```
print(“Hello, World!”);
```

To compile the *pure-dango* script, open the *pure-dango* directory, navigate to *src/scripts,* and run *installer.bat.* A prompt will appear requesting permission – click allow. Note that running installer will briefly close *File Explorer* and cause the taskbar to disappear for a few seconds. This is normal.

To run the file, either double click the .pds file, or open command prompt and run

```
cd <path to pure-dango>

node src/index.ts run <path of the file you want to run>
```

If you see the text “Hello, World!” in command prompt then, congratulations! You just wrote your first program in pure-dango! It will only get easier from here.

1.2 – print

print, a stdlib function, is the simplest way to output text in pure-dango. It takes any number of arguments, converts them to text, and prints them to the console. Each call to *print* ends with a new line.

Back when you wrote `print(“Hello, World!”);` you learned how to compile, run, and write code – but you also learned how to use print. This section is to explain the function *print*.

print takes any number of arguments. Here’s an example:

```
print(“Hi “, “friend!”);
```

This prints *Hi friend!* to command prompt. And so does:

```
print(“Hi friend!”);
```

To separate arguments, use a comma. This also applies to all other functions.

Now that you know how to output text, let's try something interesting – write a program that prints out:

1

2

3

4

5

…

But, how can we increment it each time? I mean we can use nine *print* statements but that’s tedious and doesn’t scale. To do this properly we need two things first – a way to store a number and a way to repeat an action. But how?

1.3 – Variables

A variable is a named container that can store a value. That value can be strings, numbers, or anything that the language supports. Variables are how programs remember things.

Assigning a variable:

```
<name> = <value>
```

Using a variable's value:

```
<name>
```

This will be useful later on.

Let's try using a variable. You might think this works:

```
a = 5;
```

But when you run the code, you get:

```
ReferenceError: a is not defined
```

A *ReferenceError* means we tried to use something before declaring it. So, w*e* need to define a variable first. In pure-dango, you declare variables with the keyword *new*:

1.3.1 – new

new is a keyword that declares a new variable each time it is used. The syntax of new is:

```
new <name> = <value>;
```

So, can we solve the earlier problem now?

```
new a = 1;
print(a);
a = 2;
print(a);
```

This code is starting to get redundant and clunky. We need a way to loop or increment automatically – We need something else, maybe some way to add, subtract, multiply, or divide a variable?

1.4 – Operators

Operators work just like they do in Math. There are five main operators:

1. +
2. –
3. /
4. *
5. % – modulo, gets the remainder when dividing: (2 % 100 == 2)

Operators can work on anything that is type Decimal, Number, or BigInt.

There are three types of operators: Binary, Unary, and Assignment Operators.

1.4.1 – Binary Operators

Binary Operators take two values, one on each side, and produce a result.

For example, this code snippet prints 162:

```
new a = 2 * 81;

print(a);
```

All 5 main operators are all Binary Operators.

Unlike other high-level languages, pure-dango will throw a TypeError if you try to use any operator with a string.

Here's an optional but simple program to write:

Make a program the prints:

"6 – 3 = 3"

With the number after the "=" to be using the subtraction operator.

Answer key is at Chapter 1.7, Subtraction.

This works but what if you only want one value?

1.4.2 – Unary Operators

Unary Operators take one value. The value can be on either side of the Unary Operator. Unary Operators can only be used on variables.

Unary Operators have two types:

1. Prefix – prefix is when the value is after the Unary Operator. This updates the variable before using it.
2. Postfix – postfix is when the value is before the Unary Operator. This updates the variable after using it.

There are only five Unary Operators:

1. ++ – increments the variable by one. Can be prefix or postfix
2. -- – decrements the variable by one. Can be prefix or postfix.
3. - – makes a value negative. - Is a prefix Unary Operator.
4. + – makes a standalone value being used as a condition turn in to a truthy value. Is a special prefix Unary Operator.
5. ! – checks if a value is undefined, null, negative, or any falsy value. Is a prefix Unary Operator.

To make you better understand this, I will give an example:

```
new a = 2;

print(a++);

a = 2;

print(++a);
```

This example prints:

2

3

Since the first one is a postfix and the second is a prefix.

Unary operators are great for incrementing by one, but what about adding or subtracting larger values cleanly?

1.4.3 – Assignment Operators

Assignment Operators are a shorthand for Binary Operators. Instead of writing `a = a + 2` you can write `a += 2` – they have the same result.

All assignment operators follow the same pattern: `<name> <operator>= <value>` is shorthand for `<name> = <name> <operator> <value>`. The assignment operators are: +=, -=, /=, *=, %=

Now, here's an exercise: starting with `new a = 5;`, use an assignment operator to get the result and print*:* `a – 3 = 2`

Answer key is at Chapter 1.7, Assignment

So after all we've learned from these operators can we use them for the earlier exercise to finally solve it?

Lets try it out:

```
new a = 1;
```

```
print(a++);
print(a++);
print(a++);
…
print(a++);
```

It's still so redundant but not as clunky. We need a way to loop through something with a condition.

1.5 – Loops

Loops run the *body* until the *condition* is false. A loop's condition can take Comparison Operators, Unary Operators with a value (e.g. while (!a)), and Booleans.

1.5.2 – While Loop

The while loop loops through its *body* until its condition is *false*.

The syntax of a while loop is this if it has multiple code lines inside of *body*:

```
while (condition)
{
    body
}
```

If only one code line is in the while loop you can use:

```
while (condition) body
```

1.7 – Answer Keys

Subtraction

```
print(“6 – 3 = “,  6 – 3);
```

Assignment

```
new a = 5;

a -= 3;

print(“a – 3 = “, a);
```